

2023년 1회 정보처리기사 실기 복원 문제 (해설 추가)

출처: <https://chobopark.tistory.com/372> (Life-Journey)

1번

아래 자바 코드에서 출력되는 값을 작성하시오.

```
class Static {
    public int a = 20;
    static int b = 0;
}

public class Main {
    public static void main(String[] args) {
        int a;
        a = 10;
        Static.b = a;
        Static st = new Static();
        System.out.println(Static.b++);
        System.out.println(st.b);
        System.out.println(a);
        System.out.print(st.a);
    }
}
```

정답: 10 / 11 / 10 / 20

해설:

이 문제는 자바의 **static(정적) 변수**와 **인스턴스 변수**의 차이를 묻는 문제입니다.

- `static int b`: 클래스에 딱 하나만 존재하는 변수입니다. 객체를 여러 개 만들어도 `b`는 모두가 공유하는 하나의 값입니다. `Static.b`처럼 클래스 이름으로 접근합니다.
- `public int a`: 객체(인스턴스)를 만들어야 생기는 변수입니다. 객체마다 따로 가집니다.

코드 흐름을 따라가 봅시다.

1. `a = 10`; → 지역변수 `a`는 10
2. `Static.b = a`; → static 변수 `b`에 10 저장
3. `System.out.println(Static.b++);` → **후위 증가(b++)**는 "현재 값을 먼저 사용하고, 그 다음에 1을 증가"시킵니다. 따라서 화면에는 **10**이 찍히고, 그 후 `b`는 11이 됩니다.
4. `System.out.println(st.b);` → `b`는 이제 11이므로 **11** 출력. (static 변수는 객체 `st`로 접근해도 같은 값을 봅니다.)
5. `System.out.println(a);` → 지역변수 `a`는 그대로 **10**
6. `System.out.print(st.a);` → 객체의 인스턴스 변수 `a`는 초기값 **20**

핵심: `b++`는 "출력 후 증가"라는 점, static 변수는 공유된다는 점입니다.

2번

다음 C언어의 출력값을 작성하시오.

```
#include <stdio.h>

int main(){
    char a[] = "Art";
    char* p = NULL;
    p = a;

    printf("%s\n", a);
    printf("%c\n", *p);
}
```

```
printf("%c\n", *a);
printf("%s\n", p);

for(int i = 0; a[i] != '\0'; i++)
    printf("%c", a[i]);
}
```

정답: Art / A / A / Art / Art

해설:

C언어에서 문자열은 char 배열로 저장되고, 배열 이름(a)은 그 배열의 첫 글자 주소를 의미합니다.

- char a[] = "Art"; → 메모리에 A, r, t, \0 (문자열 끝 표시) 가 저장됩니다.
- p = a; → 포인터 p가 배열 a의 시작 주소를 가리킵니다. 즉 p와 a는 같은 곳을 가리킵니다.

출력 분석:

- printf("%s\n", a); → %s 는 "주소부터 \0 까지 문자열 전체"를 출력 → **Art**
- printf("%c\n", *p); → %c 는 한 글자, *p 는 p가 가리키는 첫 글자 → **A**
- printf("%c\n", *a); → *a 도 배열 첫 글자 → **A**
- printf("%s\n", p); → p가 가리키는 문자열 전체 → **Art**
- 마지막 for문: \0 을 만날 때까지 한 글자씩 출력 → **Art**

핵심: %s 는 문자열 전체, %c 는 한 글자. 포인터 앞의 * 는 "그 주소에 든 값"을 뜻합니다.

3번

다음 C언어의 출력값을 작성하시오.

```
#include <stdio.h>

int main(){
    char* a = "qwer";
    char* b = "qwtety";

    for(int i = 0; a[i] != '\0'; i++){
        for(int j = 0; b[j] != '\0'; j++){
            if(a[i] == b[j]) printf("%c", a[i]);
        }
    }
}
```

정답: qwe

해설:

이중 for문으로 두 문자열을 비교하는 문제입니다. 바깥쪽 for는 a의 각 글자, 안쪽 for는 b의 각 글자를 돍니다. a의 한 글자가 b 안 어딘가에 있으면 그 글자를 출력합니다.

- a = "qwer" → q, w, e, r
- b = "qwtety" → q, w, t, e, t, y

하나씩 확인:

- q → b에 q 있음 → 출력 **q**
- w → b에 w 있음 → 출력 **w**
- e → b에 e 있음 → 출력 **e**
- r → b에 r 없음 → 출력 안 함

결과는 **qwe**.

핵심: 안쪽 for문이 b를 처음부터 끝까지 훑으면서 같은 글자가 있는지 검사한다는 흐름을 이해하면 됩니다.

4번

다음 괄호 안에 들어가는 용어의 Full Name 또는 약자를 작성하시오.

()은/는 비동기적인 웹 애플리케이션 제작을 위해 JavaScript와 XML을 이용한 비동기적 정보 교환 기법이다. 필요한 데이터만 웹서버에 요청해 받은 후 클라이언트에서 처리한다. Google Map과 Google pages에서 사용한 기술에 기반하여 제작되었다.

정답: AJAX (Asynchronous JavaScript and XML)

해설:

AJAX는 "Asynchronous JavaScript And XML"의 약자입니다.

- 비동기(Asynchronous)란? 화면 전체를 새로고침하지 않고, 필요한 데이터만 몰래 서버에 요청해서 받아오는 방식입니다. 예를 들어 SNS에서 스크롤만 내려도 새 글이 계속 뜨는 것이 AJAX 덕분입니다.
- 기존에는 버튼을 누르면 페이지 전체가 깜빡이며 다시 로딩됐지만, AJAX는 **일부분만 갱신**하므로 빠르고 부드럽습니다.
- 지문의 "필요한 데이터만 요청", "Google Map" 같은 키워드가 AJAX의 대표 특징입니다.

핵심 키워드: 비동기, 부분 갱신, JavaScript + XML(또는 JSON).

5번

아래 내용을 확인하여 괄호 안에 용어를 표 안에 알맞는 값을 고르시오. (보기 예: 패킷 등)

구분	내용
()	회선교환과 데이터그램 방식의 장점을 결합. 처음 패킷으로 최적 경로를 고정하고, 고정되면 그 다음은 패킷으로 나누어 고속 전송. ATM이 대표적이며 다량 데이터 연속 전송에 적합.
()	패킷교환 방식으로 동작하면서 IP 주소를 사용. 각 패킷을 정해진 경로 없이 독립적으로 처리·교환. 경로를 피해 전송 가능해 신뢰성 높음. 짧은 메시지 전송에 효과적이고 재정렬 필요.

정답: 가상회선, 데이터그램

해설:

패킷 교환 방식에는 두 종류가 있습니다.

- 가상회선(Virtual Circuit) 방식: 데이터를 보내기 전에 미리 "경로(길)"를 하나 정해 놓고, 그 길로만 모든 패킷을 순서대로 보냅니다. 연결을 미리 설정하므로 **연결 지향형**이라고 합니다. 순서가 보장되어 재정렬이 필요 없습니다. 대표 예가 **ATM**입니다.
- 데이터그램(Datagram) 방식: 경로를 미리 정하지 않고, 패킷마다 그때그때 가장 좋은 길로 따로따로 보냅니다. **비연결형**이라 빠르고 유연하지만, 패킷이 도착 순서가 뒤죽박죽이 될 수 있어 **재정렬(순서 맞추기)**이 필요합니다. **IP**가 대표적입니다.

암기 팁: "가상회선 = 미리 길 고정 = 순서 보장", "데이터그램 = 길 제각각 = 재정렬 필요".

6번

아래 내용을 확인하여 알맞는 답을 작성하시오.

- 2계층(데이터링크 계층)에서 구현되는 터널링 기술 중 하나
- L2F와 PPTP가 결합된 프로토콜로 VPN과 ISP가 이용
- IPsec과 함께 사용하면 PPTP보다 안전하지만 보안보다 익명화에 더 적합

정답: L2TP

해설:

L2TP는 "Layer 2 Tunneling Protocol"의 약자로, 이름 그대로 **2계층(데이터링크 계층)**에서 동작하는 **터널링** 프로토콜입니다.

- 터널링(Tunneling)이란? 공용 인터넷망 위에 마치 비밀 통로(터널)를 뚫어 안전하게 데이터를 보내는 기술입니다. VPN의 핵심 기술입니다.
- L2TP는 두 기존 기술인 **L2F**와 **PPTP**를 합쳐 만든 것입니다.
- L2TP 자체에는 암호화 기능이 약해서, 보통 **IPsec과 함께** 써서 보안을 강화합니다.

연관 암기: VPN 터널링 프로토콜 종류 → **PPTP, L2F, L2TP, IPsec, SSL** 등.

7번

아래 내용에서 괄호 안에 알맞는 용어를 작성하시오.

() 네트워크상의 다른 컴퓨터에 로그인하거나 원격 시스템에서 명령 실행, 파일 복사를 가능하게 하는 응용 프로그램/프로토콜. 보안 접속으로 rsh, rcp, rlogin, rexec, telnet, ftp 등을 제공하며 IP 스푸핑 방지 기능 제공. 기본 포트는 22번.

정답: SSH (Secure SHell)

해설:

SSH는 "Secure Shell"의 약자로, 원격 컴퓨터에 **안전하게(암호화하여) 접속**하기 위한 프로토콜입니다.

- 예전에는 **Telnet**으로 원격 접속을 했는데, Telnet은 비밀번호를 **평문(암호화 안 됨)**으로 보내서 도청에 취약했습니다.
- SSH는 통신 내용을 **암호화**하므로 안전합니다. 그래서 "Telnet의 보안 취약점을 보완한 것"이라는 설명이 자주 나옵니다.
- 기본 포트 번호가 22번**이라는 점은 시험에 자주 출제됩니다. (Telnet은 23번)

암기 팁: "**SSH = 22번 포트 = 암호화된 원격 접속**".

8번

아래 설명에 대한 알맞는 답을 작성하시오. (보기 예: 랜섬웨어, 스파이웨어 등)

구분	내용
(1)	감염된 시스템에서 스스로 복제해 다른 컴퓨터로 확산. 독자적으로 실행되며 다른 실행 프로그램이 필요 없음. 자가 복제 및 네트워크 전파 가능.
(2)	정상 프로그램으로 가장해 사용자가 직접 설치하도록 위장. 백도어를 만들거나 시스템을 파괴. 시스템 내부 정보를 공격자에게 빼돌림. 감염 후 스스로 복제하는 능력은 없음.
(3)	정상파일을 감염시키는 형태로 실행. 감염 대상이 없으면 실행되지 않으며 자기 복제 기능 있음. 네트워크로 전파되지 않음.

정답: 1. 웜 / 2. 트로이 목마 / 3. 바이러스

해설:

세 가지 악성코드를 **자가 복제 여부**와 **전파 방식**으로 구분하는 문제입니다.

- 웜(Worm):** 혼자서도 실행되고, **스스로 복제**해서 **네트워크를 타고** 빠르게 퍼집니다. 다른 숙주 프로그램이 필요 없는 것이 특징입니다.
- 트로이 목마(Trojan Horse):** 정상 프로그램인 척 사용자를 **속여서** 설치하게 만듭니다. **자기 복제 능력이 없다**는 것이 핵심 구분점입니다. 백도어를 열거나 정보를 빼돌립니다.
- 바이러스(Virus):** **정상 파일에 기생**해야만 동작합니다. 자기 복제는 하지만 **네트워크 전파는 못 하고**, 숙주 파일이 실행돼야 활동합니다.

구분 표:

종류	자가 복제	네트워크 전파	숙주 필요
웜	O	O	X
트로이 목마	X	X	X(위장 설치)
바이러스	O	X	O

9번

다음 아래 코드에서 이진수를 십진수로 변환하는 코드에 대해 괄호 (a)(b)의 적합한 답을 작성하시오.

```
#include <stdio.h>

int main() {
    int input = 101110;
    int di = 1;
    int sum = 0;

    while (1) {
        if (input == 0) break;
        else {
            sum = sum + (input (a)(b)) * di;
            di = di * 2;
        }
    }
}
```

```
        input = input / 10;
    }
}

printf("%d", sum);
return 0;
}
```

정답:

- 1번 풀이: (a) % , (b) 10 (또는 5, 2)
- 2번 풀이: (a) & , (b) 1

해설:

이 코드는 **2진수를 10진수로 변환**합니다. 여기서 `input=101110` 은 실제로는 십진수처럼 저장된 숫자지만, 각 자리(0,1,1,1,0,1)를 2진수의 자릿수로 취급해 변환합니다.

원리: 맨 끝 자리부터 하나씩 떼어내서 → 2의 거듭제곱(`di : 1, 2, 4, 8...`)을 곱해 더합니다.

- `input % 10` → 맨 끝 한 자리 숫자를 떼어냅니다. (예: `101110 % 10 = 0`)
- `input / 10` → 맨 끝 자리를 버립니다. (예: `101110 / 10 = 10111`)
- `di = di * 2` → 자릿값을 2배씩 키웁니다.

따라서 **(a)는 % , (b)는 10** 이면 "끝자리 추출" 방식입니다.

또 다른 풀이로 `input & 1` 을 쓰면, **비트 AND 연산**으로 맨 끝 비트만 추출할 수 있어 같은 효과가 납니다. (단 이 경우 `input` 이 진짜 2진 비트로 처리될 때 유효)

핵심: `% 10` = 끝자리 뽑기, `/ 10` = 끝자리 버리기. 이 두 연산은 자릿수 처리의 기본 패턴입니다.

10번

다음 보안 관련 설명으로 괄호 안에 알맞는 용어를 작성하시오.

()은/는 TCP/IP에서 IP 패킷을 처리할 때 발생하는 문제를 알려주는 프로토콜이다. 다른 호스트나 게이트웨이와 연결된 네트워크에 문제가 있는지 확인하기 위해 사용된다. ()을/를 이용한 공격에는 () Flooding이 있는데, ping 명령어로 () 패킷을 연속으로 보내 서버가 다른 작업을 못하게 하는 공격이다.

정답: ICMP

해설:

ICMP는 "Internet Control Message Protocol"의 약자로, 네트워크에 **문제가 생겼는지 알려주는** 메시지 전달용 프로토콜입니다.

- 우리가 인터넷 연결을 확인할 때 쓰는 **ping** 명령어가 바로 ICMP를 사용합니다. ping을 보내면 상대가 살아있는지 응답이 옵니다.
- **ICMP Flooding(=Ping of Death, Smurf 등)**: ping 패킷을 어마어마하게 많이 보내서 서버를 마비시키는 **DoS(서비스 거부) 공격**입니다.

핵심: "ping = ICMP", "문제 상황을 알려주는 프로토콜"이라는 키워드를 기억하세요.

11번

다음은 디자인 패턴에 관한 설명이다. 알맞는 답을 고르시오.

- 다른 무언가와 이어지는 인터페이스 역할을 하는 클래스
- 실제 객체를 호출하면 행위를 중간에 가로채서 다른 동작을 수행하는 객체로 변경
- 객체를 정교하게 제어하거나 객체 참조가 필요한 경우 사용
- 분리된 객체를 위임함으로써 대리 작업을 중간 단계에 삽입

정답: Proxy

해설:

프록시(Proxy) 패턴은 GoF 디자인 패턴 중 **구조 패턴**에 속합니다. "Proxy"는 "대리인"이라는 뜻입니다.

- 실제 객체에 직접 접근하지 않고, **대리 객체(Proxy)를 거쳐서** 접근하게 만듭니다.
- 이 중간 대리인이 접근을 통제하거나, 무거운 객체의 생성을 미루거나(지연 로딩), 추가 작업을 끼워 넣을 수 있습니다.
- 일상 비유: 연예인에게 직접 연락하지 않고 **매니저(대리인)**를 통하는 것과 같습니다.

암기 팁: "**Proxy = 대리인 = 중간에서 접근 제어**". (참고: 디자인 패턴은 생성/구조/행위 3분류로 나뉘며, Proxy는 구조 패턴입니다.)

12번

다음은 데이터베이스에 관련된 내용이다. 각 괄호 안에 알맞는 답을 작성하시오. (보기 예: 릴레이션 스키마 등)

구분	내용
(1)	셀 수 있는 수량의 순서 있는 열거. 테이블에서의 행을 가리키며 중복 허용 가능. 여러 객체(숫자, 문자, 배열 등)를 담을 수 있음.
(2)	어느 한 시점에 릴레이션의 내용(상태), 저장된 데이터 전체. 단순히 릴레이션 또는 릴레이션 외연이라고도 함.
(3)	특정 데이터 집합의 유니크한 값의 개수. 전체 행에 대한 특정 컬럼의 중복 수치를 나타내는 지표.

정답: 1. 튜플 / 2. 릴레이션 인스턴스 / 3. 카디널리티

해설:
관계형 데이터베이스의 기본 용어를 묻는 문제입니다. 테이블(릴레이션)을 기준으로 정리하면:

- **튜플(Tuple):** 테이블의 **한 행(Row)**. 하나의 레코드(데이터 한 건)에 해당합니다. 예: 학생 테이블에서 "홍길동, 20세, 컴퓨터과" 한 줄.
- **릴레이션 인스턴스(Relation Instance):** 어느 한 순간에 테이블에 **실제로 저장된 데이터 전체(상태)**. 데이터가 바뀌면 인스턴스도 달라집니다.
- **카디널리티(Cardinality):** 테이블의 **행(튜플)의 개수**.

추가로 함께 외우면 좋은 용어:
- **속성(Attribute)** = 열(Column)
- **차수(Degree)** = 속성(열)의 개수
- **카디널리티(Cardinality)** = 튜플(행)의 개수

암기 팁: "차수=세로(열) 개수, 카디널리티=가로(행) 개수".

13번

[학생] 테이블에서 학생 이름이 '민수'인 튜플을 삭제하는 쿼리를 작성하시오. (※ 원문에 테이블 이미지 포함)

조건: 문자열은 작은따옴표 표시, 세미콜론은 생략 가능

정답: delete from 학생 where 이름 = '민수';

해설:
데이터를 **삭제**할 때는 **DELETE** 문을 사용합니다. 기본 형식은 다음과 같습니다.

```
DELETE FROM 테이블이름 WHERE 조건;
```

- **DELETE FROM 학생** → 학생 테이블에서 삭제하겠다는 뜻
- **WHERE 이름 = '민수'** → "이름이 민수인 행만" 골라서 삭제
- 문자열 값(**민수**)은 반드시 **작은따옴표(')**로 감쌉니다.

△ 주의: **WHERE** 를 빼면 **테이블의 모든 행이 삭제**되므로 매우 위험합니다. 삭제할 대상을 **WHERE** 로 정확히 지정해야 합니다.

14번

다음 코드에서 괄호 안에 알맞는 값을 변수명으로 작성하시오. (출력값: 1 2 3 5 7 8)

```
public class Sort {
    public static void swap(int[] arr, int idx1, int idx2){
        int temp = arr[idx1];
        arr[idx1] = arr[idx2];
        arr[( 1 )] = temp;
    }

    public static void Usort(int[] array, int length){
        for (int i = 0; i < length; i++) {
            for (int j = 0; j < length - i - 1; j++) {
```

```

        if (array[j] > array[j + 1]) {
            swap(array, j, j + 1);
        }
    }
}

public static void main(String[] args) {
    int[] item = new int[] { 5, 3, 8, 1, 2, 7 };
    int nx = 6;
    Usort(item, ( 2 ));
    for (int data : item) {
        System.out.print(data + " ");
    }
}
}

```

정답: (1) idx2 / (2) nx

해설:

이 코드는 **버블 정렬(Bubble Sort)**을 구현한 것입니다. 옆에 있는 두 값을 비교해서 큰 값을 뒤로 보내는 방식입니다.

- **(1) 자리: swap 함수**는 두 값을 맞바꾸는 함수입니다.

temp = arr[idx1]; // idx1 값을 임시 보관 arr[idx1] = arr[idx2]; // idx1 자리에 idx2 값을 넣음 arr[(?)] = temp; // idx2 자리에 보관해둔 값을 넣어야 교환 완성
따라서 빈칸은 **idx2**입니다. (두 값을 서로 바꾸려면 idx2 자리에 temp를 넣어야 함)

- **(2) 자리: Usort(item, ?)** 에서 두 번째 인자는 배열의 길이(length)입니다. 배열 item의 원소가 6개이고 변수 nx = 6 이므로 빈칸은 **nx**입니다.

핵심: swap(값 교환)의 3단계 패턴 — "임시 저장 → 한쪽 덮어쓰기 → 임시값으로 나머지 채우기"를 이해하면 됩니다.

15번

다음 파이썬 코드의 알맞는 출력값을 작성하시오.

```

a = {'한국', '중국', '일본'}
a.add('베트남')
a.add('중국')
a.remove('일본')
a.update({'홍콩', '한국', '태국'})
print(a)

```

정답: {'한국', '중국', '베트남', '홍콩', '태국'} (집합이므로 순서 무관)

해설:

파이썬의 **집합(set)** 자료형을 다루는 문제입니다. 집합의 두 가지 핵심 특징을 알아야 합니다.

1. **중복을 허용하지 않음**: 같은 값을 또 넣어도 무시됩니다.
2. **순서가 없음**: 출력 순서는 의미가 없습니다.

연산 추적:

- 시작: {한국, 중국, 일본}
- add('베트남') → 베트남 추가 → {한국, 중국, 일본, 베트남}
- add('중국') → 중국은 이미 있으므로 무시 → 변화 없음
- remove('일본') → 일본 제거 → {한국, 중국, 베트남}
- update({'홍콩, 한국, 태국'}) → 여러 개를 한꺼번에 추가. 한국은 이미 있어 무시, 홍콩·태국 추가 → {한국, 중국, 베트남, 홍콩, 태국}

핵심: add (하나 추가), remove (하나 제거), update (여러 개 추가), 그리고 **중복 무시** 규칙입니다.

16번

다음 성적 테이블에서 과목별 점수의 평균이 90점 이상인 '과목이름', '최소점수', '최대점수'를 검색하는 SQL문을 작성하시오. (※ 원문에 테이블 이미지 포함)

조건: where 사용 금지, SELECT절에 별칭 사용, GROUP BY와 having 사용, 집계함수 사용

정답:

```
SELECT 과목이름, MIN(점수) AS 최소점수, MAX(점수) AS 최대점수
FROM 성적
GROUP BY 과목이름
HAVING AVG(점수) >= 90
```

해설:

그룹별 집계를 하는 SQL 문제입니다. 핵심 키워드 3개를 익히세요.

- **GROUP BY 과목이름:** 같은 과목끼리 묶습니다. "과목별로" 통계를 내고 싶을 때 사용합니다.
- **집계함수:** 묶은 그룹마다 계산하는 함수입니다.
- **MIN(점수)** = 최소점수, **MAX(점수)** = 최대점수, **AVG(점수)** = 평균점수
- **HAVING:** 그룹으로 묶은 **결과에 조건**을 걸 때 씁니다. (WHERE 는 묶기 전 개별 행에 조건을 거는 것이라 평균 같은 집계 조건엔 못 씁니다. 그래서 평균 조건은 반드시 HAVING 을 써야 합니다.)
- **AS 별칭:** **MIN(점수) AS 최소점수** 처럼 결과 열에 이름을 붙입니다.

암기 팁: "개별 행 조건 = WHERE, 그룹 조건(집계함수 조건) = HAVING". 이 구분이 자주 출제됩니다.

17번

다음 JAVA 코드에서 알맞는 출력값을 작성하시오.

```
abstract class Vehicle {
    String name;
    abstract public String getName(String val);

    public String getName() {
        return "Vehicle name: " + name;
    }
}

class Car extends Vehicle {
    public Car(String val) {
        name = super.name = val;
    }
    public String getName(String val) {
        return "Car name:" + val;
    }
    public String getName(byte val[]) {
        return "Car name:" + val;
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle obj = new Car("Spark");
        System.out.println(obj.getName());
    }
}
```

정답: Vehicle name: Spark

해설:

추상 클래스와 오버로딩(메서드 중복 정의)을 묻는 문제입니다.

- **new Car("Spark")** 로 객체를 만들 때 생성자에서 **name = val** → name에 "Spark" 저장.
- **obj.getName()** → **인자가 없는 getName()**을 호출합니다.
- Car 클래스에는 **getName(String val)** 과 **getName(byte[])** 만 있고, **인자 없는 getName()**은 부모(**Vehicle**)의 것을 그대로 물려받습니다.
- 부모의 **getName()** 은 **"Vehicle name: " + name** 을 반환 → **Vehicle name: Spark**

핵심: 메서드는 **이름이 같아도 인자(매개변수)가 다르면 다른 메서드**입니다(이것이 오버로딩). **getName()** (인자 없음)을 불렀으므로 부모의 메서드가 실행됩니

다.

18번

다음은 스키마와 관련된 내용이다. 각 괄호 안에 알맞는 답을 작성하시오.

구분	내용
(1) 스키마	사용자/응용 프로그래머가 개인 입장에서 필요한 DB의 논리적 구조 정의. 서브 스키마라고도 함. 여러 개 존재 가능하며 공유 가능.
(2) 스키마	DB의 전체적 논리적 구조. 모든 응용/사용자가 필요로 하는 데이터를 종합한 조직 전체의 DB로 하나만 존재. 접근 권한·보안·무결성 규칙 명세 정의.
(3) 스키마	물리적 저장장치 입장에서 데이터 저장 방법 기술. 레코드의 물리적 구조 정의. 시스템 프로그래머/설계자가 관리.

정답: 1. 외부 / 2. 개념 / 3. 내부

해설:
데이터베이스의 **3단계 스키마**를 묻는 문제입니다. 같은 데이터베이스를 보는 **세 가지 관점**입니다.

- **외부 스키마(External)**: 일반 **사용자/프로그래머가 보는 관점**. 각자 필요한 부분만 봅니다. 사람마다 다르므로 **여러 개** 존재합니다. "서브 스키마"라고도 합니다.
- **개념 스키마(Conceptual)**: 조직 **전체**의 논리적 구조. 모든 데이터를 종합한 것이며 **단 하나만** 존재합니다. 보안·무결성 규칙을 정의합니다.
- **내부 스키마(Internal)**: **물리적 저장** 관점. 데이터가 디스크에 실제로 어떻게 저장되는지를 정의합니다.

암기 팁: "**외부=사용자(여러 개), 개념=전체(하나), 내부=물리적 저장**". 위→아래로 추상적→구체적으로 내려갑니다.

19번

다음 아래 제어 흐름 그래프가 분기 커버리지를 만족하기 위한 테스트 순서를 쓰시오. (※ 원문에 그래프 이미지 포함)

정답: 1234561, 124567 (또는 1234567, 124561)

해설:
분기 커버리지(Branch Coverage)는 화이트박스 테스트 기법 중 하나입니다.

- **분기(Branch)**란 if문처럼 조건에 따라 길이 갈라지는 지점입니다. (참일 때 가는 길, 거짓일 때 가는 길)
- **분기 커버리지**는 모든 분기의 **참(True)과 거짓(False) 양쪽 길을 최소 한 번씩** 다 지나가도록 테스트 경로를 짜는 것입니다.
- 따라서 두 개의 경로를 제시하는데, 하나는 조건이 참일 때의 길(예: 1234561), 다른 하나는 거짓일 때의 길(예: 124567)을 포함하여 **모든 갈림길을 빠짐없이** 거치게 합니다.

참고로 커버리지 강도 순서(약→강): 문장 < 분기(결정) < 조건 < 분기/조건 < 변경조건/결정(MC/DC) < 다중조건. **분기 커버리지 = 결정 커버리지**와 같은 말입니다.

20번

다음 자바 코드에 대한 출력값을 작성하시오.

```
class Parent {
    int x = 100;
    Parent() {
        this(500);
    }
    Parent(int x) {
        this.x = x;
    }
    int getX() {
        return x;
    }
}
```

```

class Child extends Parent {
    int x = 4000;
    Child() {
        this(5000);
    }
    Child(int x) {
        this.x = x;
    }
}

public class Main {
    public static void main(String[] args) {
        Child obj = new Child();
        System.out.println(obj.getX());
    }
}

```

정답: 500

해설:

이 문제는 **생성자 호출 흐름**과 **변수 가리기(필드 은닉)**, **메서드 상속**을 동시에 묻습니다. 차근차근 봅시다.

먼저 알아야 할 개념:

- `this(...)`: **같은 클래스의 다른 생성자**를 호출합니다.
- 부모와 자식에 **같은 이름 변수 x**가 있으면 둘은 **별개의 변수**입니다(필드는 오버라이딩 안 됨). Parent의 x, Child의 x가 따로 존재합니다.
- `getX()` 는 Parent에만 정의되어 있고, 그 안의 `x` 는 **Parent의 x**를 가리킵니다.

흐름:

1. `new Child()` → `Child()` 생성자 실행 → `this(5000)` 로 `Child(int x)` 호출
2. `Child(int x)`: 그런데 자식 생성자는 실행되기 전에 **부모 생성자를 먼저** 호출합니다(묵시적 `super()`). → `Parent()` 호출
3. `Parent()` → `this(500)` → `Parent(int x)` 호출 → **Parent의 x = 500**
4. 다시 `Child(int x)`로 돌아와 `this.x = 5000` → 이건 **Child의 x = 5000** (Parent의 x와 별개!)

마지막에 `obj.getX()` 는 Parent에 정의된 메서드이고, 그 안의 x는 **Parent의 x(=500)**를 반환합니다. → **500**

핵심: `getX()`가 보는 x는 "Parent의 x"입니다. Child의 x(5000)는 영향을 주지 못합니다. **필드는 타입(선언) 기준, 메서드는 객체(실제) 기준**이라는 점을 기억하세요.